

DSA4213 Natural Language Processing for Data Science

Doudou Zhou (ddzhou@nus.edu.sg)

Lecture 2 Additional Material

- 1 Representation extensions
- 2 Word2Vec training details
- 3 Subword extension
- 4 PMI and matrix factorization
- 5 Further GloVe details
- 6 Geometry and word senses

Context can also mean a document

In the main lecture, context means a nearby word:

$X_{w,c}$ = number of times word c appears near word w .

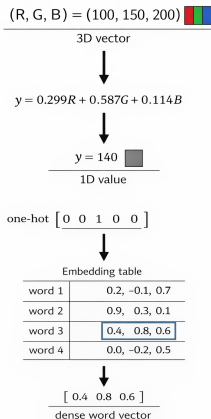
Another common construction uses a document as the context:

$X_{w,d}$ = number of times word w appears in document d .

What changes?

The rows are still words, but the columns are now documents. A row records where a word appears across the collection, so it emphasizes topical usage rather than local neighboring words.

A familiar idea: transform one representation into another

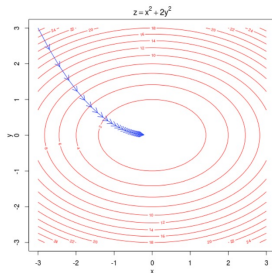


- A linear map can convert one coordinate system into another
- The output keeps selected information while changing the representation
- For words, the input is one-hot identity and the output is a learned dense vector
- Unlike the grayscale weights, the embedding coordinates are learned from language data

Parameters: one long vector θ

- θ contains **all** word vectors
- Vocabulary size $|V|$ can be huge
- Each word has **two** vectors

$$\theta = \begin{bmatrix} \mathbf{v}_{w_1} \\ \vdots \\ \mathbf{v}_{w_{|V|}} \\ \mathbf{u}_{w_1} \\ \vdots \\ \mathbf{u}_{w_{|V|}} \end{bmatrix} \in \mathbb{R}^{2d|V|}$$



Stochastic gradient descent (SGD)

- **Problem:** $J(\theta)$ sums over all windows in the corpus (potentially billions)
- Computing $\nabla_{\theta} J(\theta)$ exactly is too expensive
- **Solution:** SGD
 - ▶ sample a small number of windows (often one)
 - ▶ update parameters after each sample

```
while True:
    window = sample_window(corpus)
    theta_grad = evaluate_gradient(J, window, theta)
    theta = theta - alpha * theta_grad
```

Sparse updates with negative sampling

- Each SGD step updates only:
 - ▶ the center word vector $\mathbf{v}_c$
 - ▶ one positive context vector $\mathbf{u}_o$
 - ▶ K negative context vectors $\mathbf{u}_{w_k}$
- So the gradient is sparse in the full parameter vector θ

$$\nabla_{\theta} J_t(\theta) = \begin{bmatrix} \vdots \\ \nabla \mathbf{v}_c \\ \vdots \\ \nabla \mathbf{u}_o \\ \vdots \\ \nabla \mathbf{u}_{w_1} \\ \vdots \\ \nabla \mathbf{u}_{w_K} \\ \vdots \end{bmatrix}$$

Which Word2Vec vector do we keep?

- Training produces two matrices:
 - ▶ center/input vectors $\mathbf{v}_w$
 - ▶ context/output vectors $\mathbf{u}_w$
- Common choices after training include
 - ▶ use $\mathbf{v}_w$ as the exported word embedding;
 - ▶ use $\mathbf{u}_w$; or
 - ▶ combine them, for example $\mathbf{v}_w + \mathbf{u}_w$
- The choice is part of the representation and should be documented in an experiment

- Word-level methods treat *teach*, *teacher*, and *teaching* as separate vocabulary items
- fastText (Bojanowski et al., 2017) shares parameters through character n -grams
- A word vector is built from its subword pieces:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}(w)} \mathbf{z}_g$$

where $\mathcal{G}(w)$ contains the character n -grams and a whole-word identifier

- Benefits:
 - ▶ shares information across morphological variants;
 - ▶ improves representations of rare words; and
 - ▶ can construct vectors for many unseen word forms

Heuristic corrections to raw counts

- log transform: $\log(1 + X_{w,c})$
- cap counts: $\min(X_{w,c}, t)$
- remove or downweight stopwords
- weight nearer context words more heavily

All aim to reduce the dominance of extremely frequent events.

Association relative to independence

$$\text{PMI}(x, y) = \log \frac{P(x, y)}{P(x)P(y)}$$

For a word w and context c ,

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}.$$

A large value means that w and c occur together more often than their marginal frequencies would predict.

What PMI changes

PMI replaces raw frequency with association relative to the word and context marginals.

Given a co-occurrence matrix $\mathbf{X}$:

- Total number of co-occurrence events:

$$|D| = \sum_w \sum_c X_{w,c}$$

- Empirical probabilities:

$$\hat{P}(w, c) = \frac{X_{w,c}}{|D|}, \quad \hat{P}(w) = \frac{\sum_c X_{w,c}}{|D|}, \quad \hat{P}(c) = \frac{\sum_w X_{w,c}}{|D|}$$

- Plug-in estimate:

$$\text{PMI}(w, c) = \log \frac{\hat{P}(w, c)}{\hat{P}(w)\hat{P}(c)} = \log \frac{X_{w,c} \cdot |D|}{(\sum_{c'} X_{w,c'}) (\sum_{w'} X_{w',c})}$$

- **PPMI** (Positive PMI):

$$\text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0)$$

- Clipping keeps positive associations and avoids storing $-\infty$ for zero counts
- With K negative samples and unigram noise $q(c) = P(c)$, SGNS targets

$$\text{PMI}(w, c) - \log K.$$

A sparse explicit matrix therefore uses

$$\text{SPPMI}_K(w, c) = \max(\text{PMI}(w, c) - \log K, 0).$$

- The Word2Vec choice $q(c) \propto P(c)^{3/4}$ changes the context term; it is not only a global shift.

Example: compute PMI for one entry

Using the earlier co-occurrence matrix with window $m = 2$,

$$\text{PMI}(w, c) = \log \frac{X_{w,c} |D|}{(\sum_{c'} X_{w,c'}) (\sum_{w'} X_{w',c})}.$$

Values for (cats, chase)

$$\begin{aligned} X_{\text{cats}, \text{chase}} &= 2, \\ \sum_{c'} X_{\text{cats}, c'} &= 4, \\ \sum_{w'} X_{w', \text{chase}} &= 4, \\ |D| &= 24. \end{aligned}$$

Substitution and interpretation

$$\begin{aligned} \text{PMI}(\text{cats}, \text{chase}) &= \log \frac{2 \cdot 24}{4 \cdot 4} \\ &= \log 3 > 0. \end{aligned}$$

The pair occurs more often than expected under independence. If $X_{w,c} = 0$, PMI is $-\infty$, motivating the PPMI clipping rule.

- Build an association matrix $\mathbf{M}$ with entries:

$$M_{w,c} = \text{SPPMI}_K(w, c)$$

- Choose embedding dimension r and compute a rank- r approximation:

$$\mathbf{M} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top$$

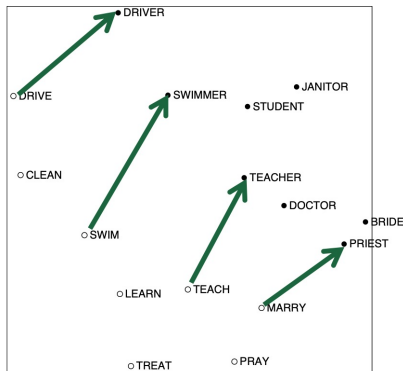
- Use rows of $\mathbf{U}_r \mathbf{\Sigma}_r$ as r -dimensional word embeddings

- Levy and Goldberg (2014) analyze the dot products learned by SGNS
- For noise distribution $q(c)$ and K negatives, the target association is

$$\mathbf{u}_c^\top \mathbf{v}_w \approx \log \frac{P(w, c)}{P(w)q(c)} - \log K.$$

- If $q(c) = P(c)$, this reduces to $\text{PMI}(w, c) - \log K$
- Computationally:
 - ▶ SGNS learns from sampled pairs with SGD;
 - ▶ SPPMI-SVD constructs and factorizes an explicit matrix.

Low-rank embeddings can reveal recurring directions



- After appropriate weighting + low-rank approximation, semantic structure can emerge
- Example: linear “meaning components” in the vector space

Explicit and predictive methods can be competitive

Method	PMI- log k	SPPMI	SVD			SGNS		
			$d = 100$	$d = 500$	$d = 1000$	$d = 100$	$d = 500$	$d = 1000$
$k = 1$	0%	0.00009%	26.1%	25.2%	24.2%	31.4%	29.4%	7.40%
$k = 5$	0%	0.00004%	95.8%	95.1%	94.9%	39.3%	36.0%	7.13%
$k = 15$	0%	0.00002%	266%	266%	265%	7.80%	6.37%	5.97%

Table 1: Percentage of deviation from the optimal objective value (lower values are better). See 5.1 for details.

WS353 (WORDSIM) [13]		MEN (WORDSIM) [4]		MIXED ANALOGIES [20]		SYNT. ANALOGIES [22]	
Representation	Corr.	Representation	Corr.	Representation	Acc.	Representation	Acc.
SVD (k=5)	0.691	SVD (k=1)	0.735	SPPMI (k=1)	0.655	SGNS (k=15)	0.627
SPPMI (k=15)	0.687	SVD (k=5)	0.734	SPPMI (k=5)	0.644	SGNS (k=5)	0.619
SPPMI (k=5)	0.670	SPPMI (k=5)	0.721	SGNS (k=15)	0.619	SGNS (k=1)	0.59
SGNS (k=15)	0.666	SPPMI (k=15)	0.719	SGNS (k=5)	0.616	SPPMI (k=5)	0.466
SVD (k=15)	0.661	SGNS (k=15)	0.716	SPPMI (k=15)	0.571	SVD (k=1)	0.448
SVD (k=1)	0.652	SGNS (k=5)	0.708	SVD (k=1)	0.567	SPPMI (k=1)	0.445
SGNS (k=5)	0.644	SVD (k=15)	0.694	SGNS (k=1)	0.540	SPPMI (k=15)	0.353
SGNS (k=1)	0.633	SGNS (k=1)	0.690	SVD (k=5)	0.472	SVD (k=5)	0.337
SPPMI (k=1)	0.605	SPPMI (k=1)	0.688	SVD (k=15)	0.341	SVD (k=15)	0.208

Table 2: A comparison of word representations on various linguistic tasks. The different representations were created by three algorithms (SPPMI, SVD, SGNS) with $d = 1000$ and different values of k .

Source: Levy and Goldberg (2014)

No single representation dominates every benchmark; explicit SPPMI can be competitive with SGNS under matched choices.

Review: probability ratios isolate useful contexts

For target words `ice` and `steam`, compare how strongly the same context x associates with each:

$$R_x = \frac{P(x \mid \text{ice})}{P(x \mid \text{steam})}.$$

	$x = \text{solid}$	$x = \text{gas}$	$x = \text{water}$	$x = \text{random}$
$P(x \mid \text{ice})$	large	small	large	small
$P(x \mid \text{steam})$	small	large	large	small
R_x	large	small	≈ 1	unstable

- `solid` and `gas` distinguish the two target words in opposite directions
- `water` is associated with both and therefore does not distinguish them
- a rare unrelated context provides no stable discriminative signal

Goal: encode ratio information using vector differences.

- GloVe makes a modeling assumption:

$$\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j \approx \log X_{ij}$$

- Rewrite co-occurrence counts as conditional probabilities:

$$P(j \mid i) = \frac{X_{ij}}{\sum_k X_{ik}} \Rightarrow \log P(j \mid i) = \log X_{ij} - \log \sum_k X_{ik}$$

- When taking differences, normalization terms cancel:

$$\tilde{\mathbf{w}}_x^\top (\mathbf{w}_a - \mathbf{w}_b) \approx \log \frac{P(x \mid a)}{P(x \mid b)}$$

Notation:

- X_{ij} : number of times word j appears in the context window of word i
- $X_{i.} := \sum_k X_{ik}$: total context counts for center word i (row sum)

Step 1: counts $\rightarrow$ conditional probability

$$P(j \mid i) := X_{ij}/X_{i.}$$

Step 2: take logs

$$\log P(j \mid i) = \log (X_{ij}/X_{i.}) = \log X_{ij} - \log X_{i.}$$

Step 3: build a ratio by subtracting two log-probabilities

For two center words a and b :

$$\log \frac{P(x \mid a)}{P(x \mid b)} = \log P(x \mid a) - \log P(x \mid b)$$

Step 4: expand and simplify (normalization cancels)

$$\begin{aligned} \log P(x \mid a) - \log P(x \mid b) &= (\log X_{ax} - \log X_{a.}) - (\log X_{bx} - \log X_{b.}) \\ &= (\log X_{ax} - \log X_{bx}) - (\log X_{a.} - \log X_{b.}) \end{aligned}$$

Recall from previous slide:

$$\log \frac{P(x | a)}{P(x | b)} = (\log X_{ax} - \log X_{bx}) - (\log X_a - \log X_b.)$$

GloVe modeling assumption:

$$\log X_{ij} \approx \mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j$$

Substitute into the ratio expression:

$$\log X_{ax} - \log X_{bx} \approx (\mathbf{w}_a - \mathbf{w}_b)^\top \tilde{\mathbf{w}}_x + (b_a - b_b)$$

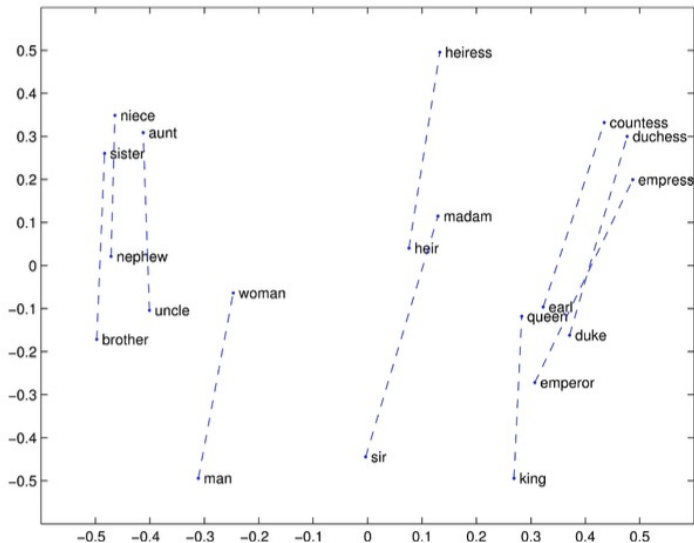
Absorb row-normalization into biases:

$$(b_a - b_b) \approx \log X_a - \log X_b.$$

Approximate relation:

$$\tilde{\mathbf{w}}_x^\top (\mathbf{w}_a - \mathbf{w}_b) \approx \log \frac{P(x | a)}{P(x | b)}$$

Vector differences can encode recurring relations



One vector mixes the senses of “pike”

The same static vector $\mathbf{v}_{\text{pike}}$ must represent:

- a sharp point or staff;
- a type of elongated fish;
- a railroad line or road;
- the future (“coming down the pike”); and
- to withdraw from doing something (Australian English).

- Under the paper's generative model, a standard word embedding approximately mixes sense vectors

$$v_{\text{pike}} \approx \alpha_1 v_{\text{pike}_1} + \alpha_2 v_{\text{pike}_2} + \alpha_3 v_{\text{pike}_3}$$

- The weights are approximately the probabilities of the senses:

$$\alpha_i \approx P(\text{sense } i \mid \text{pike}), \quad \sum_i \alpha_i = 1$$

- **Surprising result:**

- ▶ Under model and sparsity assumptions, sparse coding can approximately recover sense vectors

Example: sparse components recovered for “tie”

tie				
trousers	season	scoreline	wires	operatic
blouse	teams	goalless	cables	soprano
waistcoat	winning	equaliser	wiring	mezzo
skirt	league	clinching	electrical	contralto
sleeved	finished	scoreless	wire	baritone
pants	championship	replay	cable	coloratura

The recovered components separate clothing, sports, equality, wiring, and music-related uses.